

HANDWRITTEN DIGIT RECOGNIZER

Convolutional Neural Network (CNN) using PyTorch

Project Report • Machine Learning / Computer Vision

1. Project Overview

This project develops an end-to-end handwritten digit recognition system using a Convolutional Neural Network (CNN) implemented with PyTorch. The model learns visual features from 28×28 grayscale handwritten digit images and classifies them into ten classes, from 0 to 9.

The workflow covers dataset inspection, image preprocessing, CNN architecture design, model training, validation, performance analysis, prediction on an unlabeled test set, and saving the trained model for future inference.

2. Dataset & Preparation

Dataset	Images	Labels
Training	3,000	Available
Validation	10,000	Available
Test	10,000	Unlabeled

- Image dimensions: 28 × 28 pixels, grayscale.
- Training set is balanced: 300 images for each digit 0–9.
- Pixel values are scaled from 0–255 to 0–1.
- Images are reshaped to 1 × 28 × 28.
- Test labels are -1, so test accuracy cannot be calculated; validation data is used for evaluation.

3. Technology Stack

Python • PyTorch • NumPy • Matplotlib • Scikit-learn • Seaborn • Pillow • Google Colab

4. CNN Model Architecture

The CNN extracts increasingly meaningful visual patterns through convolution and pooling, then maps the learned representation to ten digit classes.

Layer	Configuration
Input	1 × 28 × 28 grayscale image
Convolution 1	32 filters, 3×3 kernel, padding=1 + ReLU
Max Pooling 1	2×2
Convolution 2	64 filters, 3×3 kernel, padding=1 + ReLU
Max Pooling 2	2×2
Flatten	64 × 7 × 7 features
Fully Connected	128 neurons + ReLU
Dropout	0.5
Output	10 classes (0–9)

5. Training Configuration

- Loss function: CrossEntropyLoss.
- Optimizer: Adam.
- Learning rate: 0.001.
- Batch size: 64.
- Training duration: 10 epochs.
- GPU is used automatically when available.

6. Model Evaluation

The model was evaluated on 10,000 labeled validation images. The project generates a classification report containing precision, recall, F1-score and support for every digit, along with a confusion matrix to identify class-level errors.

- Training and validation performance were visualized across epochs.
- Misclassified validation images were extracted for qualitative error analysis.
- The confusion matrix shows strong concentration along the diagonal, indicating that most validation samples were classified correctly.

7. Real-Image Inference

A separate handwritten image named digit_5.jpg was resized to 28×28, converted to grayscale, normalized and passed through the trained CNN. The demonstration predicted digit 5 with a displayed confidence of 99.79%.

8. Model Saving & Reuse

The trained weights were saved as `handwritten_digit_cnn.pth`. Saving the state dictionary allows the model to be restored later without repeating the training process.

```
model = DigitCNN().to(device)
model.load_state_dict(torch.load("handwritten_digit_cnn.pth", map_location=device))
model.eval()
```

9. Project Workflow

Dataset Loading → Data Exploration → Preprocessing → DataLoaders → CNN Construction → Loss & Optimizer → Training → Validation → Classification Report → Confusion Matrix → Misclassification Analysis → Test Prediction → Model Saving → New Image Prediction

10. Key Outcomes

- Built a complete CNN-based computer vision pipeline in PyTorch.
- Trained the model using 3,000 labeled handwritten images.
- Evaluated generalization on 10,000 labeled validation images.
- Generated class-wise precision, recall and F1-score metrics.
- Used a confusion matrix and misclassified samples for error analysis.
- Generated predictions for 10,000 unlabeled test images.
- Demonstrated inference on a new handwritten digit image.
- Saved the trained model for future reuse without retraining.

11. Repository & Reproducibility

The GitHub repository contains the Jupyter/Colab notebook, trained `.pth` model, README, requirements file and visual result files. The raw `.p` dataset files are intentionally not included in the public repository.

GitHub: https://github.com/skm-1718/HANDWRITTEN_DIGIT_RECOGNIZER_CNN

12. Future Enhancements

Potential extensions include data augmentation, batch normalization, learning-rate scheduling, hyperparameter tuning, semi-supervised learning using unlabeled data, and deployment through a Streamlit web application.